Memory Tagging Under Pressure: Hunting an MTE Ordering Race in iOS 26.x

From kernelcache static analysis to physical-device stress testing

K. Ercihan

Switch - Computer Emergency Response Team

06.11.2026

Switch

Summary

1. `id && hostnamectl | head -n 3 #whoAmI`
2. Acknowledgements
3. Motivation
4. Static Reverse Engineering
5. The Virtualization Detour
6. Physical-Device Testing
7. Results and Interpretation
8. Research Materials

Switch_

```
id && hostnamectl | head -n 3
#whoAmI
```

Switch_

Kaya Ercihan

- Security Engineer @ SWITCH
- Advanced malware analysis across macOS, Linux, Android, and iOS
- Product and low-level vulnerability research, reverse engineering, and exploit development
  kernels, browser engines, document-processing software, speculative execution, and CPU microcode
- Operate the organization's MISP threat-sharing platforms
- Phishing-kit analysis, extract compromised customer data for affected clients' IR
- Proactive OT-security threat mitigation for critical energy, logistics, and manufacturing



Switch

Acknowledgements

Idea Contributions and Review

Idea and methodology contributions

Slavo Greminger

Provided the technical input behind `PENDING_FLOOD`: combining live Mach ports with queued Mach messages before the FLOOD phase, adding pending-message pressure as a separate allocator-state test.

Tobias Ospelt

Suggested the prefill-based test and background-execution methodologies, which became `PREFILL_FLOOD` and the `BGProcessingTask`-based experiments.

Review and feedback

Andreas Renz

Reviewed the initial static-analysis report and provided helpful technical feedback.

Thank you for the ideas and review that helped shape the experimental methodology.

Switch_

Motivation

The Question That Started the Research

Which kernel function is **security-critical**?

↓

Which feature is new enough that its implementation may be **less mature**?

↓

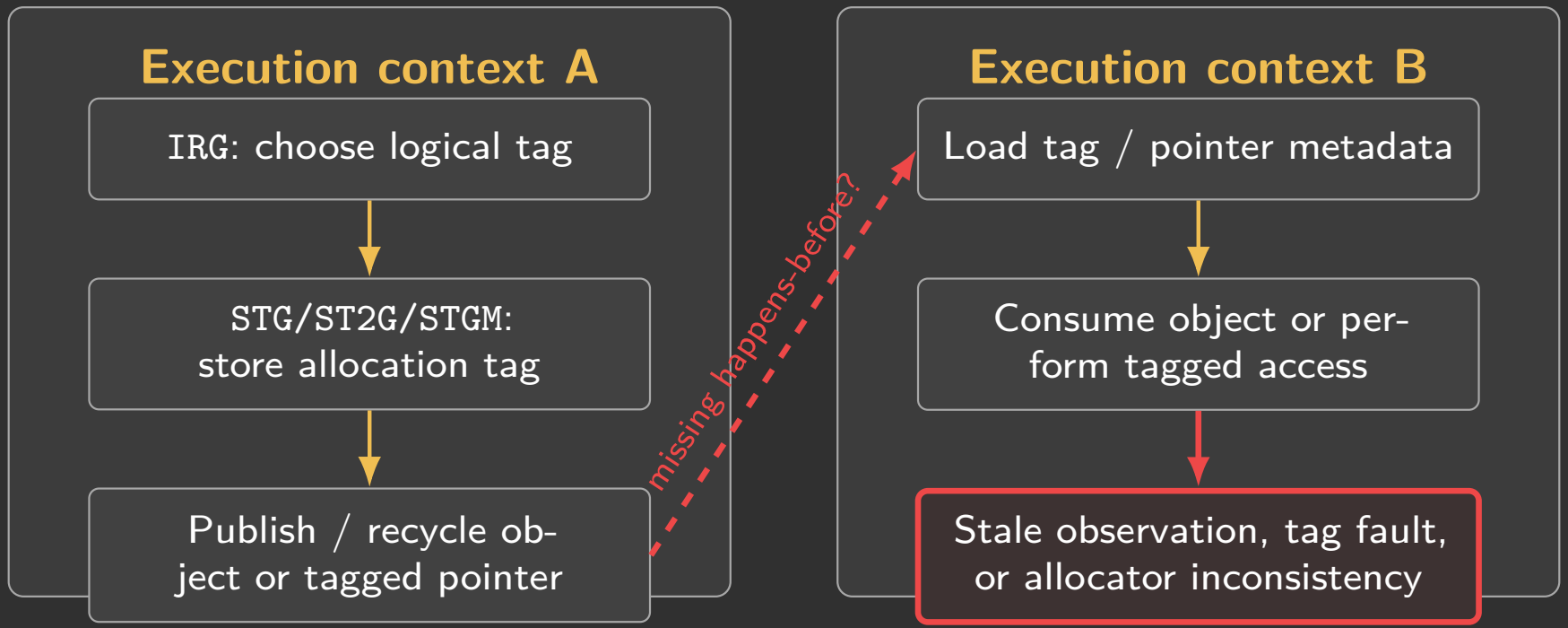
Memory tagging looked like an especially interesting boundary.

- Would it be fun to find a **race condition** in that path?
- It would also be a good reason to **reverse an iOS kernel** and follow the evidence wherever it led.
- So the starting point was deliberately exploratory: find a security-sensitive primitive, formulate a concurrency hypothesis, and see where the journey goes.

The initial motivation was curiosity, not a pre-existing vulnerability claim.

Switch_

What Would a TOCTOU Mean in This Path?

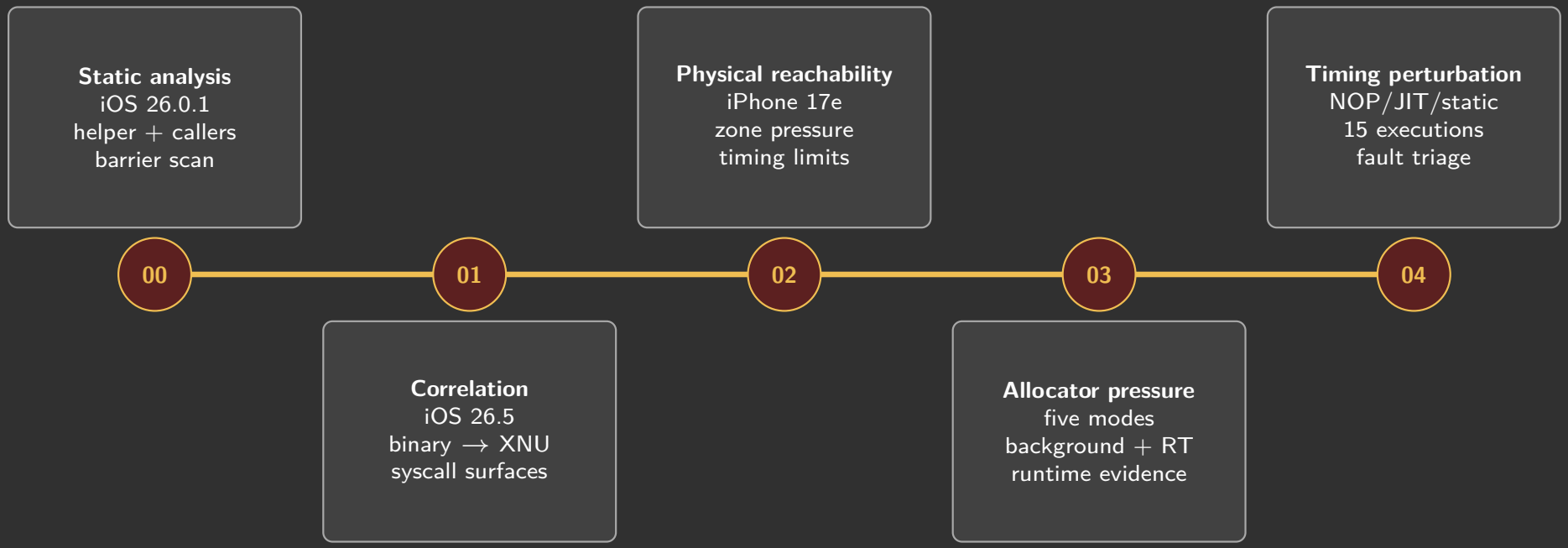


The hypothesis is **not** that `IRG` or `STG` is incorrect. The question is whether tag stores are ordered with later publication, recycling, and consumption under concurrency.

Research framing from Reports 00–04 [1–5].

Switch_

The Five-Paper Research Journey



Each report narrowed a different uncertainty: where the helper is, how userland reaches it, which surface survives lock analysis, and what black-box testing can establish.

Series continuity summarized in Report 04 [5].

Switch_

Static Reverse Engineering

The MTE Primitive in One Slide



logical tag embedded in pointer top byte

```
IRG X0, X0, X2
// choose random logical tag
```

```
STG / ST2G / STGM
// write allocation tags to granules
```

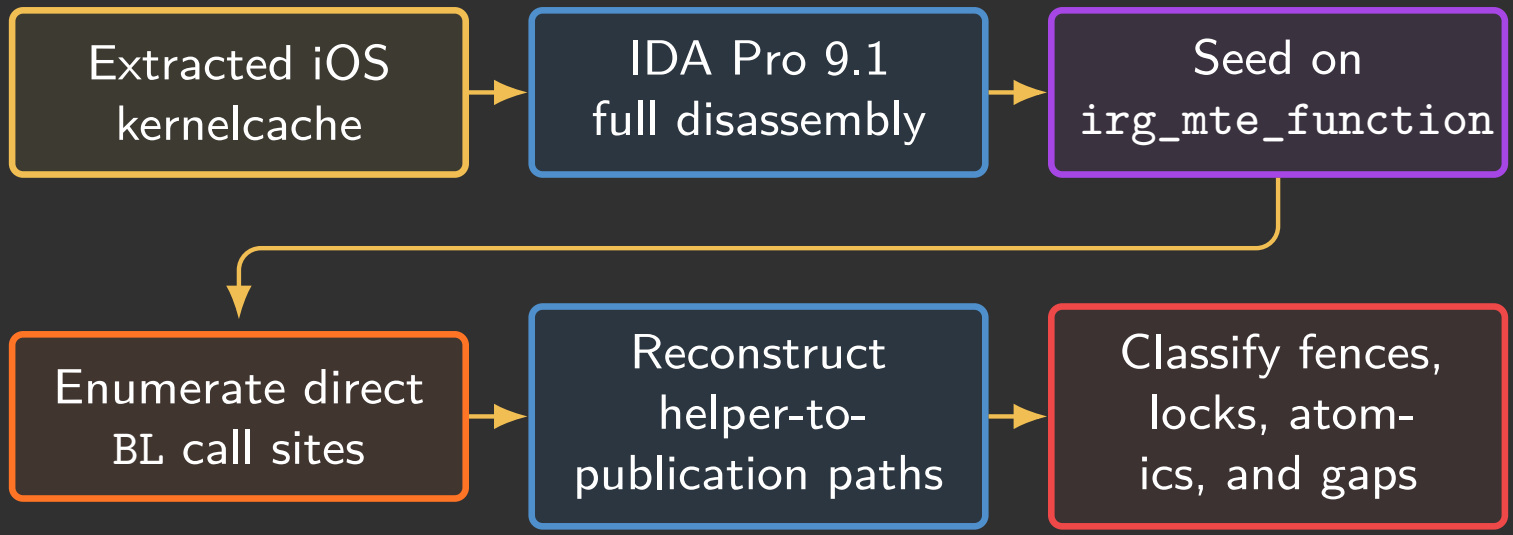
- The caller supplies an address, size, and exclude mask.
- `IRG` chooses a logical tag; `STG`, `ST2G`, and `STGM` store allocation tags.
- Userland does **not** choose the final kernel tag. It controls allocator pressure and object lifetime transitions.
- MTE instructions are not general-purpose memory-ordering fences.

A correct tag value is insufficient if publication and consumption are not ordered.

Reports 00–02 and Arm architectural terminology [1–3, 6].

Switch_

Static Analysis Workflow



1. Dump helper
`dump_current_func_instructions.py`
Complete instruction stream.

2. Enumerate + scan
`ida_irg_toctou_check.py`
Callers and synchronization primitives.

3. Classify paths
`verify_barrier_paths.py`
Per-caller ordering result.

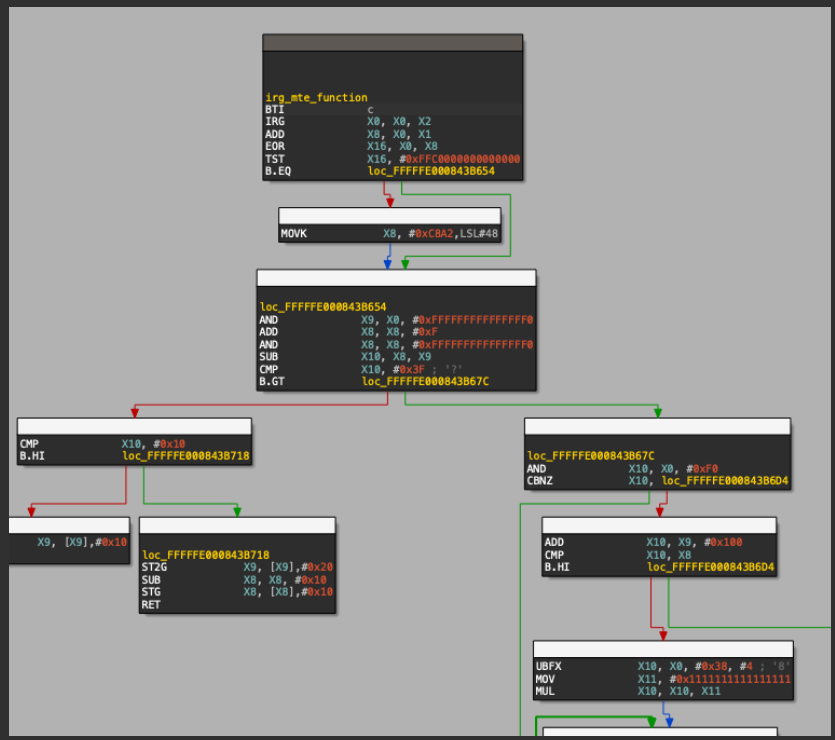
Methodology from Report 00 [1].

Switch_

Seed Function: Tag Generation Without an Explicit Fence

```
1  FFFFFE000A8578A4: BTI   c
2  FFFFFE000A8578A8: IRG   X0, X0, X2
3  FFFFFE000A8578AC: AND   X9, X0, #...FFF0
4  FFFFFE000A8578B0: ADD   X8, X1, X0
5  ...
6  FFFFFE000A8578D4: STG   X9, [X9], #0x10
7  FFFFFE000A8578FC: STGM  X11, [X9]
8  FFFFFE000A857934: ST2G  X9, [X9], #0x20
9  FFFFFE000A857978: ST2G  X9, [X9], #0x20
10 FFFFFE000A857988: ST2G  X9, [X9], #0x20
11 FFFFFE000A857974: RET
```

- Architectural MTE operations are present.
- No visible `DMB`, `DSB`, or `ISB` in the helper body.
- Publication ordering must come from a caller, a higher-level lock, or a complete acquire/release protocol.

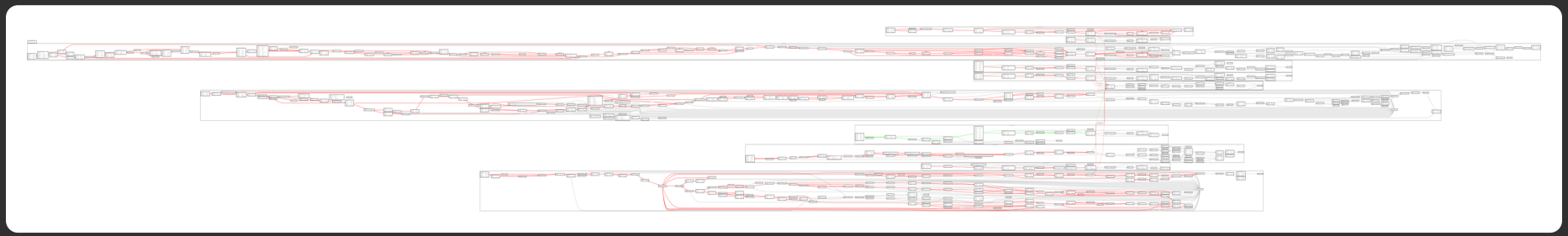


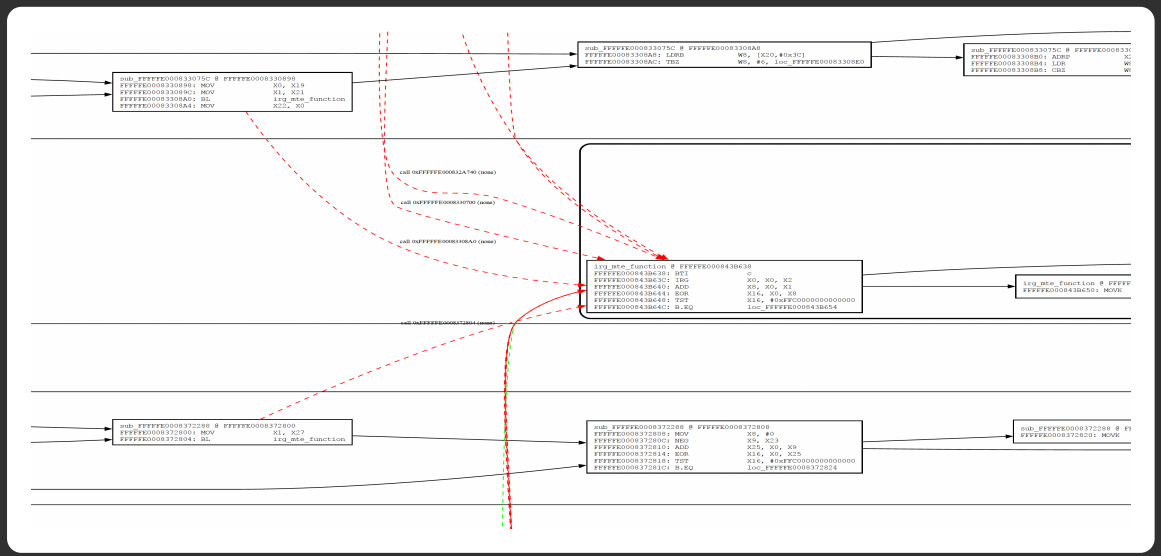
Zoomed IDA control-flow graph of `irg_mte_function`.

Report 00, Fig. 2; iOS 26.5 helper sequence in Report 01 [1, 2].

Switch_

Caller Graph: From One Seed to the Surrounding Kernel





12 direct BL sites

11 caller functions

The graph was the pivot from one unnamed helper to path-level synchronization analysis and later source-level correlation.

Caller graph artifact and Report 00 [1].

Switch_

Static Findings and Runtime Gates

1 caller context
explicit `DMB ISH`

↓

Remaining traced paths
no `DMB`/`DSB`/`ISB` across the helper-to-publication segment

↓

Atomic RMWs were common
`CASA`, `CASL`, `LDADD`, `LDADDA`

Acquire/release-qualified atomics can contribute to ordering, but their presence alone did not identify an end-to-end publication protocol.

Follow-up Tag Propagation Across Additional Regions

1. The caller saves the tagged pointer returned in `X0`.
2. `LDRB [ctx,+0x3C]` reads a flag from the caller's context structure. `TBZ ..., #6` decides whether the additional tagging path is entered.
3. If enabled, a global count controls how many times the follow-up tag-store worker is executed.

These are control-flow gates for **additional tagging work**, not proof of a race. The TOCTOU hypothesis still requires concurrent publication or reuse without sufficient synchronization.

Report 00, caller classification and trigger preconditions [1].

Switch_

Correlating Binary Functions to Public XNU

```
1  ADRL X2, aAllocCountWrap
2  ; "alloc_count wrap-around"
3  BL   sub_FFFFFE000AED2BF8
4
5  ADRL X2, off_FFFFFE0007C88F70
6  ; "hw_lck_ticket_lock"
7  BL   sub_FFFFFE000A6C8848
```

String xrefs supplied semantic anchors inside otherwise unnamed functions.

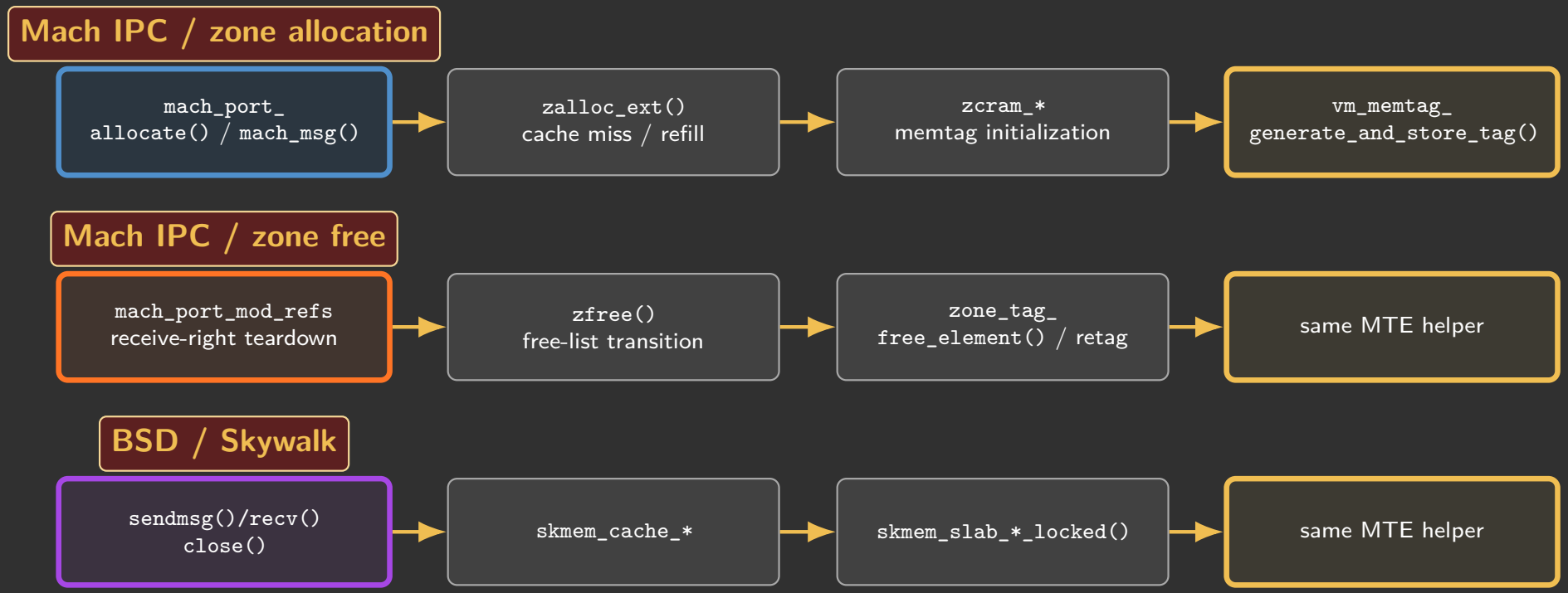
1. Locate stable diagnostic, panic, lock, and allocator strings.
2. Follow xrefs to recover function boundaries and local call neighborhoods.
3. Compare control flow, arguments, constants, structure offsets, and neighboring calls with public XNU.
4. Require multiple independent anchors before accepting a mapping.

`irg_mte_function`
⇓
`vm_memtag_generate_and_store_tag()`

String anchors in the caller-graph artifact; source mappings summarized in Report 01 [2, 7].

Switch_

Correlation Graph: Userland to the MTE Helper

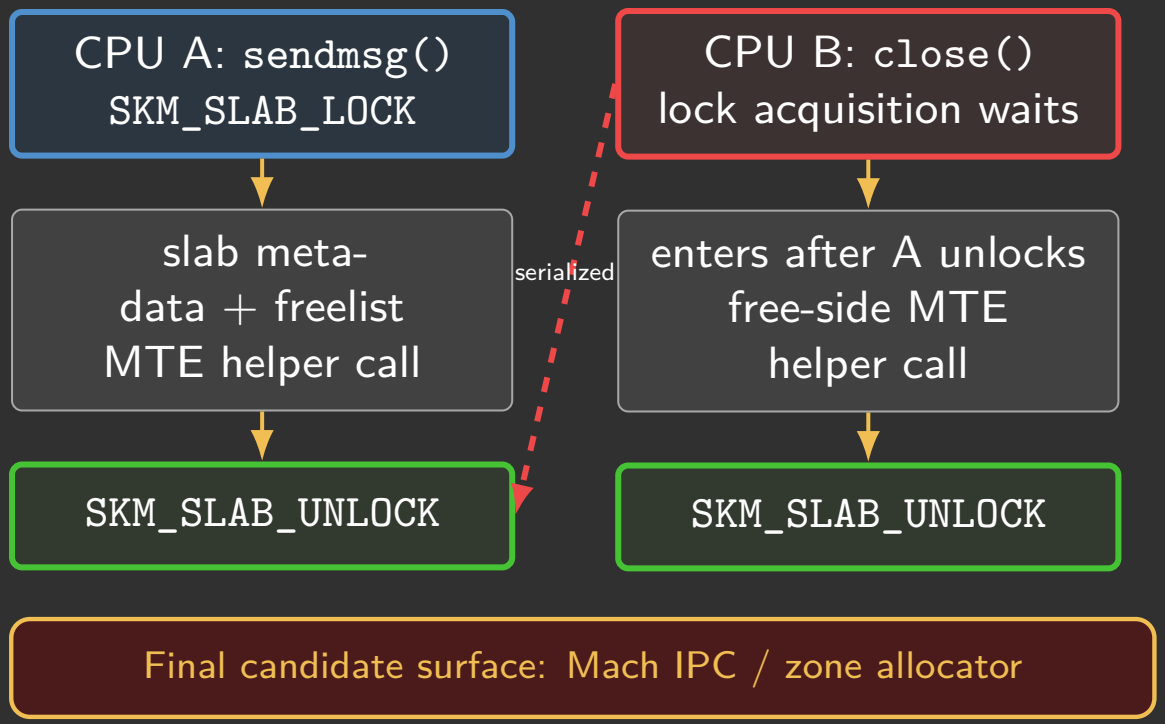


Among the correlated families, two userland-relevant pressure surfaces were investigated in depth: Mach IPC / zone and BSD socket / Skywalk. Userland controlled object pressure and lifetime transitions, not the generated tag.

Correlation matrix and syscall chains from Report 01 [2].

Switch_

Why Skywalk Was Removed from the Final Race Surface



Assessment

- Allocation and free bodies execute under per-cache `skm_sl_lock`.
- Same-cache MTE critical sections cannot overlap.
- Skywalk remains a reachability/noise control, not the final race surface.

Skywalk serialization analysis in Reports 02–03 [3, 4].

Switch_

The Virtualization Detour

First Dynamic Idea: Reuse the PCC Leak Environment

PCC looked attractive because it promised an iOS-like kernel, repeatable instrumentation, and direct observation of the suspected `IRG` / tag-store boundary.

Same PCC image identical setup

Mach-O contains real `IRG`

```
$ xcrun llvm-objdump -d
 -mattr=+mte ./testirg
 | grep -i irg
./testirg:  file format mach-o arm64
100004078:  94000071  bl
 0x10000423c
  <_irg_x0_x0_real_or_emulated>
000000010000423c
<_irg_x0_x0_real_or_emulated>:
10000424c:  9adf1000
 irg x0, x0
...
```

`0x9adf1000` encodes `IRG X0,X0`.

M3 laptop **failed**

M5 laptop **worked**

Host-dependent execution outcome

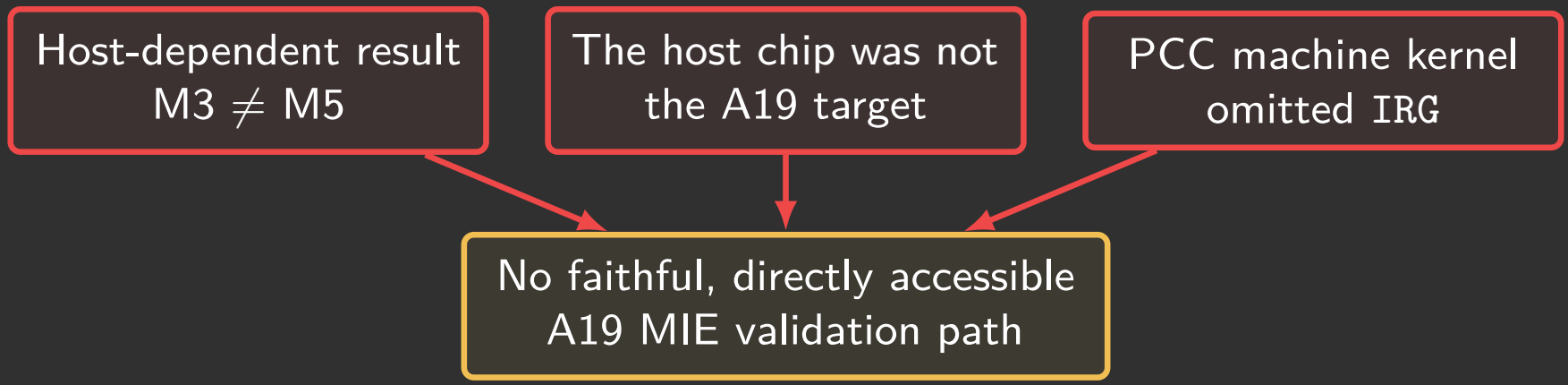
Identical guest setup produced different M3/M5 results:
this was not self-contained A19 emulation.

Interpretation: `IRG` was tunneled to the host Apple-silicon cryptographic co-processor path, not an A19 model.

Supplied PCC emulation experiment; not part of Reports 00–04.

Switch_

Why Virtualization Could Not Validate the iPhone 17 Hypothesis



Even a hypothetical accurate A19 co-processor model would not help when the supplied machine kernel does not expose the omitted `IRG` path to the experiment.

Decision

- Reject virtualization as the validation substrate.
- No Apple Security Research Device; no Corellium access under the available budget.
- Buy an iPhone 17e and pivot to a stock-app physical-device methodology.

Virtualization findings supplied for this talk; physical-device pivot documented in Reports 02–04 [3–5].

Switch_

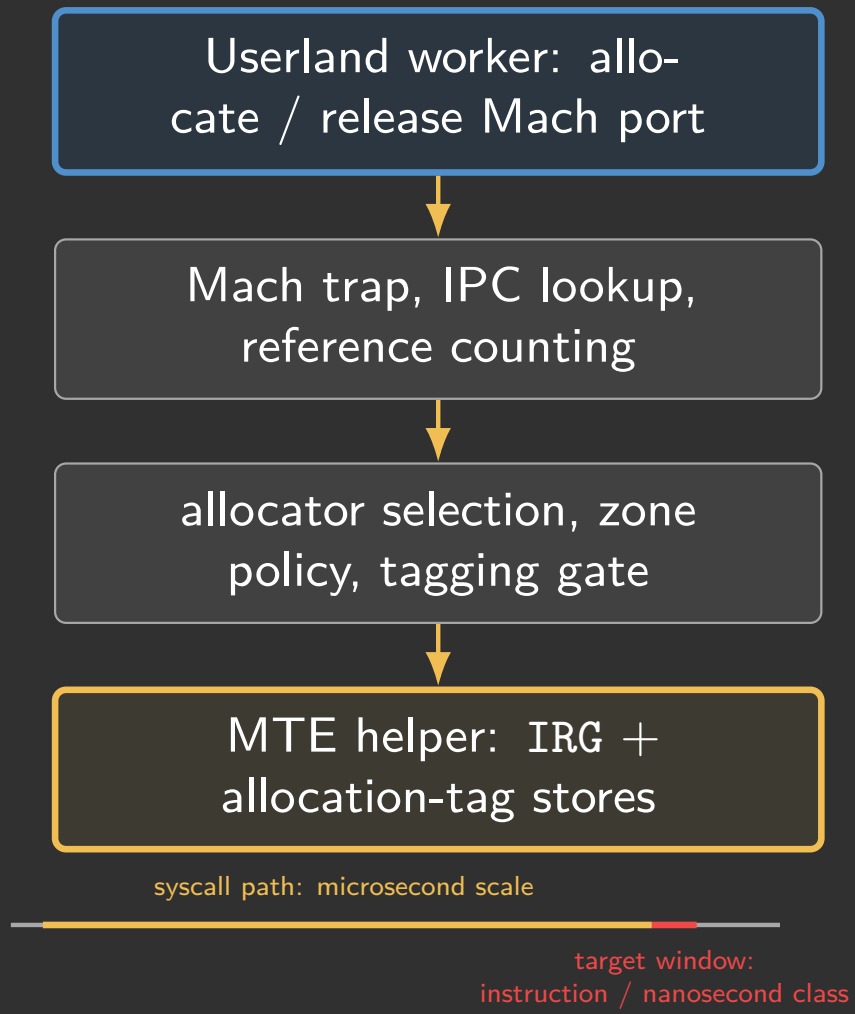
Physical-Device Testing

Physical Target and the Stock-Userland Constraint

Concrete setup

- **iPhone 17e**, iOS 26.5.1
- `MTETrigger.app`: C backend through SwiftUI
- Standard low-priority application container
- Primary surface: Mach IPC / zone allocator
- Skywalk: sequential reachability/noise control only

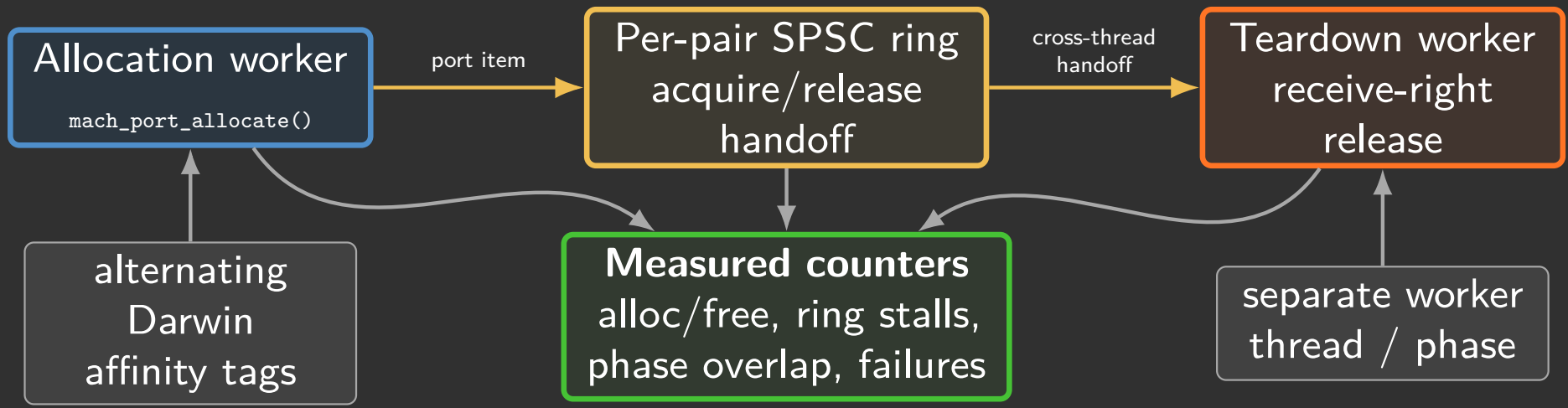
The app displays the static helper mapping, but cannot trace helper entry/exit, element address, or executing CPU.



Physical target and timing limitation from Report 02 [3].

Switch_

Harness Architecture: Force Lifetime Churn, Measure What Is Observable



Observable: `g_overlap_count` is an approximate indicator that allocation and free userland phases were concurrently active.

Not observable: helper execution, physical CPU, same-element reuse, overlap duration, or `STG` → `vm_memtag_load_tag()` overlap.

Harness design and counter semantics from Reports 02–04 [3–5].

Switch_

Implemented Pressure Methodologies

FAST
receive-right alloc/free only
maximize transitions

PIPELINE
receive + send right,
message, teardown
semantic coverage

FLOOD
one-port SPSC handoff
tight same-class churn

PREFILL_FLOOD
live reserve, then FLOOD
occupancy / reuse state

PENDING_FLOOD
queued messages,
then FLOOD
ipc_kmsg pressure

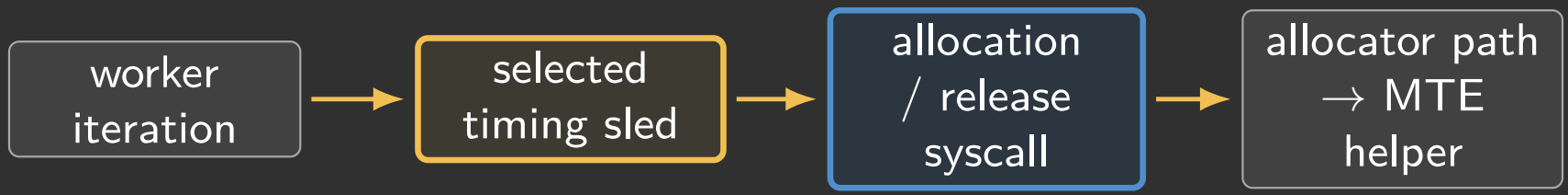
BGProcessingTask
same C trigger
in background
scheduler / life-
cycle context

The methods vary transition rate, object state, message pressure, and scheduler context while preserving the same core zone-allocation/free hypothesis.

Implemented modes and scientific purpose from Report 03 [4].

Switch_

Timing Perturbation: Move Syscall Arrival Times, Not the Helper Itself



Loop NOP
Deterministic repeating delay with loop overhead.

MAP_JIT pool
Random callable lengths; pure or mixed ARM64 only when JIT is available.

Static signed pool
Random selection from 29 precompiled functions containing 1–10000 NOPs.

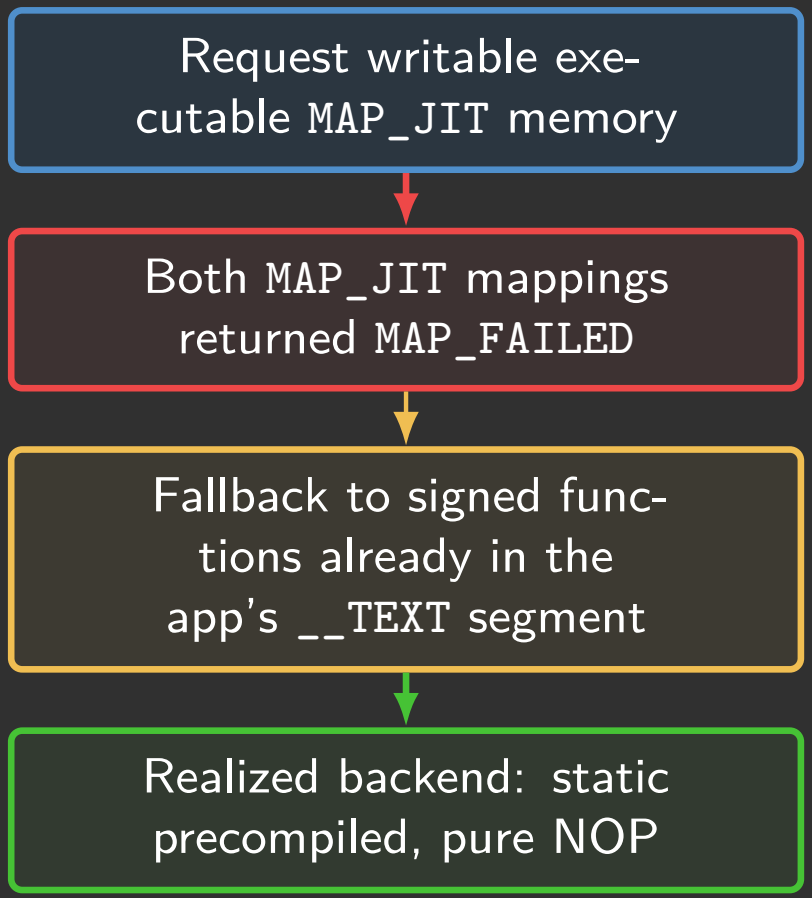
No sled can be inserted between `IRG` and `STG/ST2G/STGM`; timing is perturbed only before selected Mach syscalls.

PIPELINE jittered only allocation-side syscall entry; message sending and teardown remained non-jittered.

Timing-layer design and insertion coverage from Report 04 [5].

Switch_

Why the Instruction Sleds Had to Be Precompiled



- The process requested a runtime `MAP_JIT` region, but both mapping attempts returned `MAP_FAILED`.
- Consequently, no writable JIT region was available in which generated instructions could later be executed.
- The static fallback worked because its functions were already compiled into the app's signed `__TEXT` segment.
- Therefore the requested "mixed" JIT configuration was realized as precompiled, pure-NOP sleds.

Requested mixed-JIT mode $\neq$ executed static pure-NOP backend

MAP_JIT denial and signed fallback from Report 04 [5].

Switch_

The Fifteen-Execution Campaign

Mode	A: Baseline	B: RT option	C: 64-sled jitter request
FAST	executed	executed; RT path not invoked	executed
PIPELINE	executed	executed; RT path not invoked	executed
FLOOD	executed	RT requested	executed
PREFILL_FLOOD	executed	RT requested	executed
PENDING_FLOOD	executed	RT requested	executed

Five pressure modes were each run once under baseline, RT-labelled, and JIT-jitter-labelled configurations.

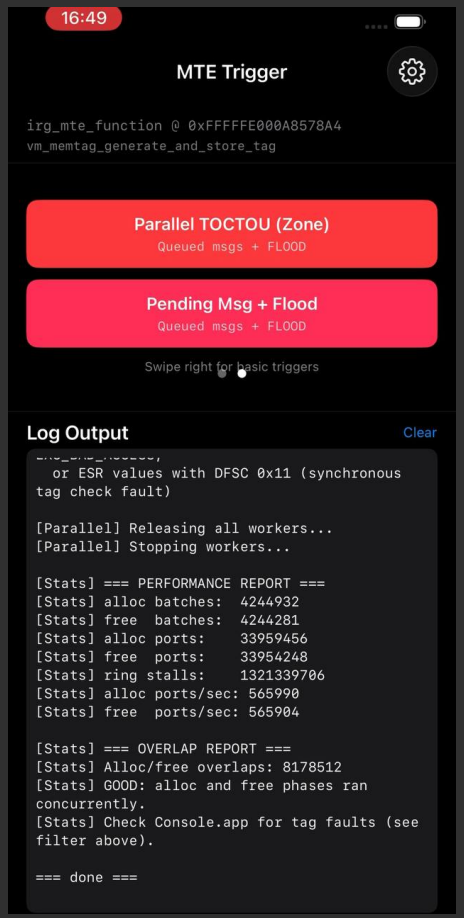
Each configuration was tested once. RT and jitter were not combined, and the jitter runs used the static pure-NOP fall-back.

Campaign design and effective RT scope from Report 04 [5].

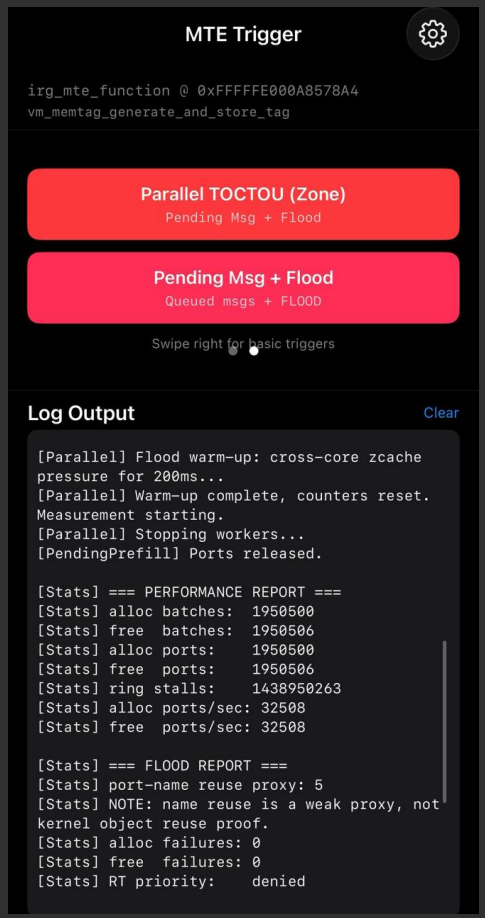
Switch_

Results and Interpretation

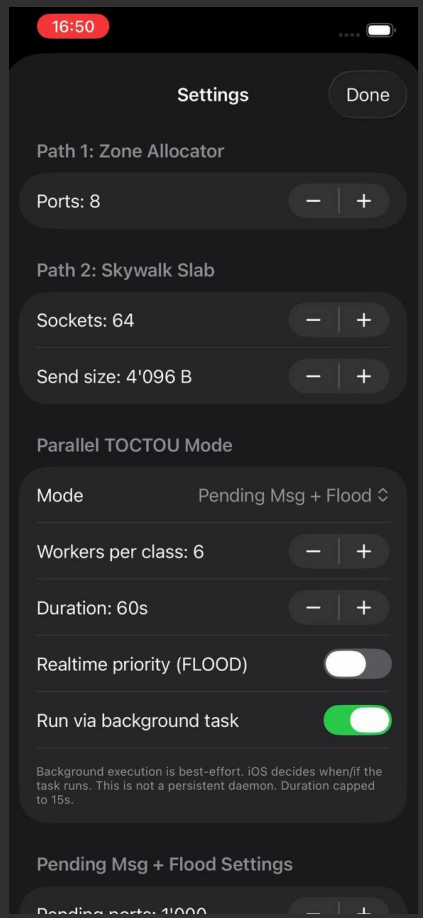
Physical-Device Run Evidence and Settings



High-throughput batched zone pressure



Pending-message FLOOD



Background-task settings

Foreground and background evidence from Report 03, Figs. 1–3 [4].

What the Harness Demonstrated Quantitatively

33,959,456
allocated ports
high-throughput run

565,990
allocated ports/s

8,178,512
phase-overlap
increments

1000 + 1000
pending ports +
queued messages
zero pending failures

12,443,103
overlap increments
early zone-only run

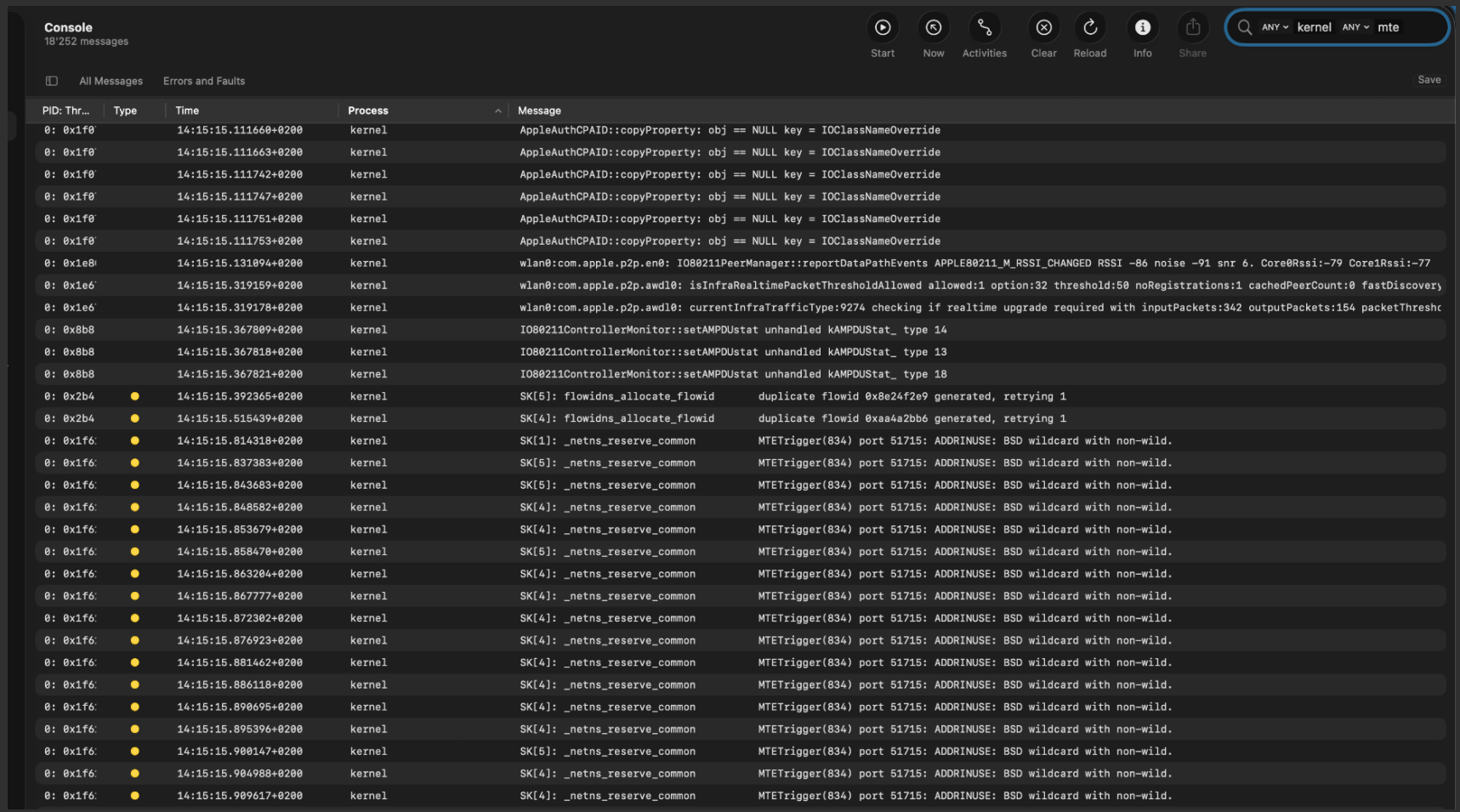
279,493
background allocations
158,445 overlaps

The physical device sustained millions to tens of millions of kernel-object lifetime transitions. That is strong pressure and reachability evidence, but still only a proxy for the same-object instruction window.

Observed run counters from Reports 02–03 [3, 4].

Switch_

Fault Triage: No External MTE Signature Observed



Switch

Fault Triage: Search Terms and Classification

Exact Console search terms

`"DFSC 0x11"`	`EXC_BAD_ACCESS`
`"synchronous tag check fault"`	`"Data Abort"`
`"asynchronous tag check fault"`	`panicString`
`"tag check fault"`	`"panic(cpu"`
`"tag mismatch"`	`TFSR`
`TAG_FAULT`	`TFSREO`

0

external fault matches

Acceptance rule: a positive match had to originate from the kernel, a crash reporter, or a panic report and include actual fault context. Text printed by `MTETrigger` as search guidance was excluded.

`DFSC 0x11` triage

Total matching rows	15
From `MTETrigger`	15
External fault rows	0

Classification: no external MTE-fault manifestation was observed. This does not prove that the helper was skipped or that the candidate race is absent.

Fifteen-run Console triage and exact filter set from Report 04 [5].

Switch_

What Was Proven – and What Remains Unresolved

Supported by the evidence

- Helper/caller mapping and MTE instruction sequence.
- Zone and Skywalk userland-to-XNU correlation.
- Same-cache Skywalk serialization under `skm_sl_lock`.
- Physical Mach IPC churn and phase overlap.
- Foreground/background execution and static signed NOP fallback.

Not demonstrated

- Direct runtime observation of the helper or its CPU ID.
- Same-element reuse from free-side retag to allocation return.
- `STG` visibility overlapping `vm_memtag_load_tag()` or a tagged access.
- Stale-tag race, allocator failure, panic, or exploitability.

The missing evidence was not more phase overlap; it was same-object instruction-window correlation.

Evidence boundaries and conclusions from Reports 02–04 [3–5].

Switch_

Next Validation Stage and Final Takeaways

Minimum useful kernel-side trace tuple

1. zone name + element address
2. CPU ID + helper entry/exit timestamps
3. free-side retag + freelist transition
4. allocation return + later `vm_memtag_load_tag()`
5. same-object subsequent use or fault outcome

1. Static analysis found a credible ordering question.

2. Strings and structure mapped the binary graph to XNU and reachable syscalls.

3. Virtualization was not faithful enough; physical testing was necessary.

4. The harness proved reachability and pressure, not the race.

5. The negative result is bounded fault evidence, not a refutation.

Best next step: SRD/debug kernel or another controlled instrumentation environment. Black-box follow-up should add randomized repetitions, RT+JIT combinations, a pure-NOP control, and complete per-run metadata.

Recommended next validation in Reports 03–04 [4, 5].

Switch_

Research Materials

Memory Tagging Under Pressure: Hunting an MTE Ordering Race in iOS 26.x

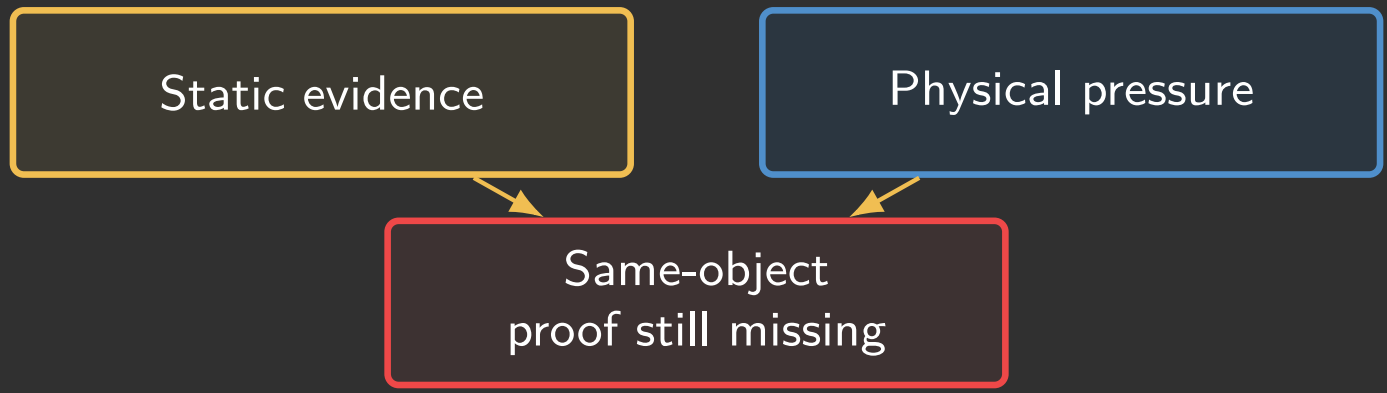


https://github.com/ercihan/a19_mie

Switch_

Thanks for your attention!

Questions and discussion



Sources I

1. Ercihan, K. *Investigating a Potential TOCTOU in the iOS 26.0.1 MTE Tagging Path.* Tech. rep. Report 00 (Independent Security Research, Mar. 2026).
2. Ercihan, K. *iOS 26.5 MTE Tagging Paths: Caller–Trigger Correlation and TOCTOU Preconditions.* Tech. rep. Report 01 (Independent Security Research, June 2026).
3. Ercihan, K. *Physical-Device Evaluation of a Potential TOCTOU in the iOS MTE Tagging Path on iPhone 17e / iOS 26.5.1.* Tech. rep. Report 02 (Independent Security Research, June 2026).
4. Ercihan, K. *Dynamic Pressure Validation for Potential iOS Kernel MTE Allocator-State Races.* Tech. rep. Report 03 (Independent Security Research, June 2026).
5. Ercihan, K. *Dynamic Timing-Perturbation Validation for Potential iOS Kernel MTE Allocator-State Races.* Tech. rep. Report 04 (Independent Security Research, Aug. 2026).

Switch_

Sources II

6. Arm Limited. *Arm Architecture Reference Manual for A-profile Architecture.* Architectural reference for IRG, GMI, STG, ST2G, STGM and DMB/DSB/ISB (2026).
7. Apple Inc. *XNU source: vm_memtag, zone allocator, and Skywalk slab code.* Source-level reference points used for reverse mapping. 2026.

Switch_